IPSSI

2e année Bachelor Informatique

Mathématiques appliquées à l'Intelligence Artificielle

Fiche de synthèse

Jour 2 — Optimisation

Auteur : Mohamed EL AFRIT

www.mohamedelafrit.com

Année académique : 2025–2026

© 2025 Mohamed EL AFRIT — IPSSI

Ce contenu est distribué sous licence

Creative Commons BY-NC-SA 4.0

https://creativecommons.org/licenses/by-nc-sa/4.0/deed.fr

1 Fonction de coût (loss function)

La **fonction de coût** $J(\mathbf{w})$ mesure l'erreur globale du modèle. Pour la régression linéaire, on utilise l'**erreur quadratique moyenne** (MSE) :

Fonction de coût MSE

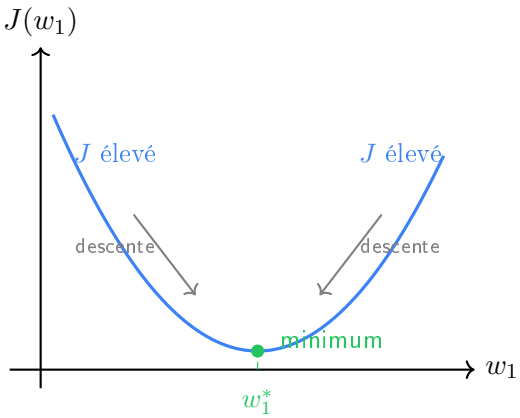
$$J(\mathbf{w}, b) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 = \frac{1}{n}\sum_{i=1}^{n}(y_i - (\mathbf{w}^T\mathbf{x}_i + b))^2 \tag{1}$$

Le coût dépend des **paramètres** $(\mathbf{w}, b)$, pas des données (fixées).

Objectif

Apprendre = trouver $\mathbf{w}^* = \arg\min_{\mathbf{w}} J(\mathbf{w})$, c'est-à-dire les poids qui **minimisent** l'erreur.

Interprétation graphique — dans le cas simple (1 paramètre w_1, biais fixé), $J(w_1)$ est une **parabole** (courbe en U). Le minimum est le creux de la courbe :



2 Dérivée — résumé

Dérivée

La dérivée de f en x mesure le **taux de variation instantané** :

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h} \tag{2}$$

2.1 Dérivées utiles

Fonction $f(x)$	Dérivée $f'(x)$	Exemple
c (constante)	0	$f = 5 \Rightarrow f' = 0$
$ax + b$	a	$f = 3x + 2 \Rightarrow f' = 3$
x^2	$2x$	$f = x^2 \Rightarrow f' = 2x$
ax^2	$2ax$	$f = 4x^2 \Rightarrow f' = 8x$
$(x-a)^2$	$2(x-a)$	$f = (x-3)^2 \Rightarrow f' = 2(x-3)$
Règle de la chaîne : $[g(h(x))]' = g'(h(x)) \cdot h'(x)$		

2.2 Lien dérivée – minimum

À retenir

Au **minimum** d'une fonction : $f'(x^*) = 0$ (pente nulle, fond de la vallée). Pour le MSE (fonction **convexe**), le seul point où $J' = 0$ est le minimum global.

3 Descente de gradient

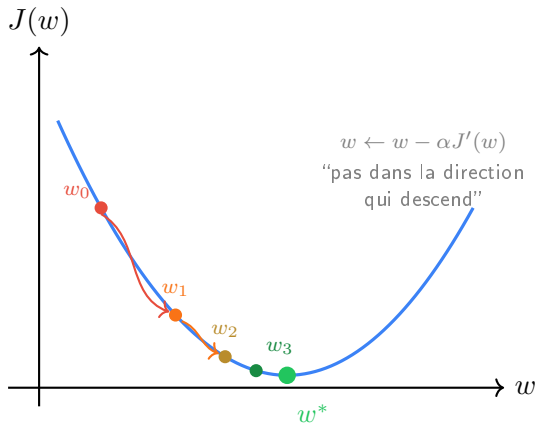
Algorithme de la descente de gradient

Répéter jusqu'à convergence :

$$\boxed{w \leftarrow w - \alpha \cdot \frac{\partial J}{\partial w}} \tag{3}$$

1. **Prédire** : $\hat{y}_i = \mathbf{w}^T \mathbf{x}_i + b$ pour tout i
2. **Calculer l'erreur** : $J(\mathbf{w})$ (MSE)
3. **Calculer le gradient** : $\nabla J = \left(\frac{\partial J}{\partial w_0}, \frac{\partial J}{\partial w_1}, \ldots\right)$
4. **Mettre à jour** : $\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla J$

Illustration — itérations successives sur la courbe de coût :



Analogie

Descendre une montagne **les yeux bandés** : on tâte la pente (gradient), on fait un pas dans la direction descendante ($-\alpha\nabla J$), on recommence.

4 Taux d'apprentissage α

Le taux d'apprentissage α contrôle la **taille des pas** :

Valeur de α	Comportement	Conséquence
Trop petit (ex : 10^{-5})	Convergence très lente	Des milliers d'itérations nécessaires. Pas pratique.
Bon (ex : 10^{-2})	Convergence rapide et stable	J diminue régulièrement puis se stabilise. Idéal.
Trop grand (ex : 1)	Oscillation ou divergence	On "saute" par-dessus le minimum. J augmente !

Choisir α

Commencer par $\alpha = 0{,}01$, observer la courbe J vs itérations : si elle descend trop lentement $\Rightarrow$ augmenter ; si elle oscille ou monte $\Rightarrow$ diminuer.

5 Application à la régression linéaire

Pour $\hat{y}_i = w_1 x_i + w_0$ et $J = \frac{1}{n}\sum(y_i - \hat{y}_i)^2$, les **dérivées partielles** sont :

$$\frac{\partial J}{\partial w_1} = \frac{-2}{n}\sum_{i=1}^{n} x_i \cdot (y_i - \hat{y}_i) \tag{4}$$

$$\frac{\partial J}{\partial w_0} = \frac{-2}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i) \tag{5}$$

Mise à jour simultanée à chaque itération :

$$w_1 \leftarrow w_1 - \alpha\frac{\partial J}{\partial w_1} \qquad w_0 \leftarrow w_0 - \alpha\frac{\partial J}{\partial w_0}$$

Résultat sur le dataset salaire

Départ : $w_1 = 0$, $w_0 = 0$, $\alpha = 0{,}001$, 5 000 itérations.

Descente de gradient : $\hat{y} = 2{,}0672\,x + 31{,}2830$ (MSE = 20,46)
Formule analytique : $\hat{y} = 2{,}0672\,x + 31{,}2830$ (MSE = 20,46)

Les deux méthodes convergent vers le **même résultat**. La descente de gradient est plus lente mais **universelle**.

 CC BY-NC-SA 4.0

6 Formules essentielles à retenir

Formules du Jour 2

$$\text{Fonction de coût} \quad J(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{6}$$

$$\text{Dérivée} \quad f'(x) = \lim_{h\to 0}\frac{f(x+h)-f(x)}{h} \tag{7}$$

$$\text{Mise à jour (GD)} \quad w \leftarrow w - \alpha \cdot \frac{\partial J}{\partial w} \tag{8}$$

$$\text{Gradient pente} \quad \frac{\partial J}{\partial w_1} = \frac{-2}{n}\sum x_i(y_i - \hat{y}_i) \tag{9}$$

$$\text{Gradient biais} \quad \frac{\partial J}{\partial w_0} = \frac{-2}{n}\sum (y_i - \hat{y}_i) \tag{10}$$

$$\text{Condition de minimum} \quad \nabla J(\mathbf{w}^*) = \mathbf{0} \tag{11}$$

7 Tableau récapitulatif

Concept	Formule	Code Python
Fonction de coût	$J = \frac{1}{n}\sum(y_i - \hat{y}_i)^2$	`np.mean((y - y_pred)**2)`
Dérivée de $(w-a)^2$	$2(w-a)$	`2 * (w - a)`
Gradient $\partial J/\partial w_1$	$\frac{-2}{n}\sum x_i e_i$	`-2/n * np.sum(x * err)`
Gradient $\partial J/\partial w_0$	$\frac{-2}{n}\sum e_i$	`-2/n * np.sum(err)`
Mise à jour	$w \leftarrow w - \alpha\nabla J$	`w -= alpha * grad`
Taux d'apprentissage	α (ex : 0,01)	`alpha = 0.01`
Prédiction	$\hat{y} = w_1 x + w_0$	`y_pred = w1*x + w0`
Boucle d'entraînement	T itérations	`for _ in range(5000):`

Boucle du Jour 2 : Prédiction $\hat{y}$ $\longrightarrow$ Erreur J $\longrightarrow$ Gradient ∇J $\longrightarrow$ Mise à jour $\mathbf{w}$ $\circlearrowleft$

© 2025 Mohamed EL AFRIT — IPSSI • www.mohamedelafrit.com • CC BY-NC-SA 4.0